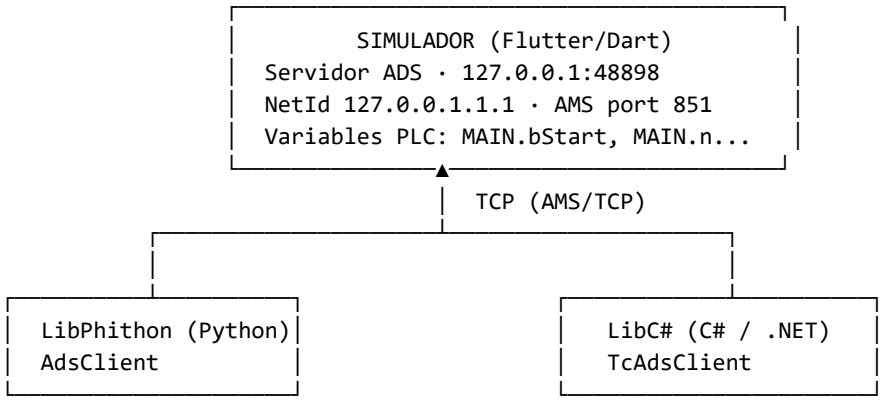
1. Arquitectura y proceso general

1.1. Objetivo

Disponer de dos librerías de comunicaciones **totalmente independientes entre sí** —una en Python y otra en C#— que implementen el protocolo **ADS (Automation Device Specification) **de Beckhoff, junto con un** simulador** de dispositivo (PLC) hecho en **Flutter/Dart** que permite probar ambas librerías sin necesidad de hardware TwinCAT real.

Las tres piezas comparten un único **contrato de interoperabilidad**: el archivo `PROTOCOL.md`. Gracias a ese contrato, cualquier cliente (Python o C#) puede hablar con el simulador Dart, y ambas librerías producen exactamente los mismos bytes en la red.

1.2. Piezas del sistema



- **Simulador (servidor)**. Escucha conexiones TCP, interpreta los mensajes ADS y responde. Mantiene el estado de un PLC ficticio: variables con nombre (`MAIN.*`), handles, imagen de proceso y notificaciones.
- **LibPhithon (cliente Python)**. Librería pura (solo biblioteca estándar) que permite conectar, leer, escribir y suscribir notificaciones desde Python.
- **LibC# (cliente C#/.NET)**. Librería .NET 6 sin dependencias externas con la

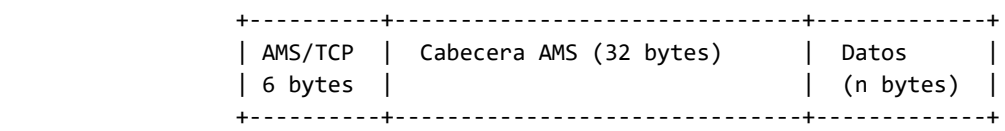
misma funcionalidad, orientada a aplicaciones .NET.

1.3. Capas de cada implementación

Todas las implementaciones siguen la misma división en capas:

1. **Transporte AMS/TCP**. Empaqueta cada mensaje con una cabecera de 6 bytes (2 reservados + 4 de longitud) seguida de la cabecera AMS de 32 bytes y los datos.
2. **Cabecera AMS**. Campos de enrutado (NetId y puerto de origen/destino), `command_id`, `state_flags`, longitud, código de error e `invoke_id`.
3. **Comandos ADS**. Operaciones: leer/escribir datos, leer información del dispositivo, leer/cambiar estado, leer-y-escribir, comandos sum (múltiples accesos en una transacción) y notificaciones.
4. **Codificación de tipos PLC**. Conversión entre los tipos IEC 61131-3 (`BOOL`, `INT`, `REAL`, `STRING`, `ARRAY`, …) y sus bytes little-endian.
5. **API pública**. Capa de alto nivel: `read_by_name(...)`, `write_by_name(...)`, `add_device_notification(...)`, etc.

1.4. Formato de cable (resumen)



La cabecera AMS/TCP:

Offset	Tamaño	Campo	Valor
0	2	Reservado	`0x0000`
2	4	Longitud	bytes de cabecera AMS + datos

La cabecera AMS:

Offset	Tamaño	Campo
0	6	`target_netid`
6	2	`target_port`
8	6	`source_netid`

14	2	source_port
16	2	command_id
18	2	state_flags
20	4	length
24	4	error_code
28	4	invoke_id

El servidor responde **intercambiando** `target` ↔ `source`, manteniendo `command_id` e `invoke_id`, y poniendo `state_flags = 1` (o `4` para notificaciones).

1.5. Flujo de una transacción (ejemplo de lectura)

6. El cliente resuelve el tamaño de la variable (`size`) a partir del tipo PLC o de la información de símbolo.
7. El cliente construye un comando `READ` (0x0002) con `index_group = 0xF005` (`SYM_VALBYNAME`), el nombre de la variable y el tamaño a leer.
8. El cliente envía el frame AMS/TCP por el socket.
9. El simulador parsea la cabecera, localiza el símbolo por nombre y devuelve sus bytes (con `error_code = 0`).
10. El cliente descodifica los bytes según el tipo PLC y devuelve el valor.

El proceso completo (diseño → implementación → pruebas → despliegue) se describe en la guía de integración.

1.6. Decisiones de diseño

- **Independencia total**: cada librería no conoce la existencia de la otra; solo comparten el contrato `PROTOCOL.md` (y el simulador como punto de prueba).
- **Sin dependencias externas**: Python usa la biblioteca estándar; C# usa solo la BCL de .NET 6; Dart usa solo el SDK (`dart:io`, `dart:typed_data`, …). Esto facilita compilar y ejecutar en entornos sin red.
- **Little-endian** en todos los campos enteros, tal y como exige ADS.
- **Concurrencia segura**: cada cliente protege la escritura del socket con un mutex, usa `invoke_id` monotónico y un hilo/tarea lector que casa respuestas y

reparte notificaciones.